Payment Intelligence Ecosystem

Nền tảng dữ liệu cho theo dõi độ ổn định thanh toán
và chấm điểm rủi ro gian lận

Báo cáo thực tập (6 tuần)

Đỗ Trung Tuyến

Mã nguồn: `https://github.com/TrungTuyenDo02/Payment_Intelligence_Ecosystem`

Tháng 11/2026

Tóm tắt

Đề tài xây dựng một nền tảng dữ liệu nhỏ cho mảng thanh toán, chạy hoàn toàn trên máy cá nhân, và trên đó hai sản phẩm trả lời hai câu hỏi của vận hành:

- **Product 01 — Độ ổn định thanh toán:** hệ thống có đang chạy ổn không? Dữ liệu sự kiện thanh toán *giả lập* có cài sẵn sự cố, KPI theo khung 15 phút và bộ phát hiện bất thường.
- **Product 02 — Rủi ro gian lận:** giao dịch này có đáng nghi không? Mô hình chấm điểm trên dataset công khai IEEE-CIS (590.540 giao dịch, 3,5% gian lận).

Điểm nhấn của đề tài là **đánh giá trung thực**: dữ liệu chia theo thời gian, mọi bước học chỉ dùng tập huấn luyện, tập kiểm tra chỉ được mở *một lần* cho mỗi cấu hình đã chốt, và mọi lần chạy đều tái lập được.

Kết quả chính: Logistic Regression đạt PR-AUC 0,165 trên tập test (gấp khoảng 4,8 lần đoán ngẫu nhiên; baseline luật 0,078). Bộ phát hiện bất thường có tính đến số lượng giao dịch bắt được 2/4 sự cố trên bộ đánh giá độc lập, không báo nhầm lần nào trong 7 ngày; bộ cơ sở đơn giản bắt 0/4 và báo nhầm 14 lần. Gradient Boosting đạt PR-AUC test 0,408, gấp khoảng 2,5 lần Logistic Regression, recall 0,51 tại FPR 5%. Thí nghiệm rò rỉ cho thấy mã hoá theo nhãn có dùng nhãn validation làm PR-AUC tăng 0,063; tiền xử lý trên mọi dòng và khoảng trống thời gian không tạo khác biệt đo được; riêng chia ngẫu nhiên lại cho kết quả *thấp hơn*, chưa giải thích hết.

Cách đọc báo cáo. Mục 2 mô tả hệ thống. Mục 3 là kiến thức nền, viết ngắn gọn; các kỹ thuật ít phổ biến được giải thích kỹ trong khung màu xanh. Mục 4–6 là kết quả, mỗi con số kèm mã lần chạy (`run-…`, `det-…`) để truy lại trong repo. Mục 7–8 là hạn chế và hướng phát triển.

Mục lục

1 Giới thiệu **3**

1.1 Bối cảnh và câu hỏi . . . 3

1.2 Mục tiêu và ràng buộc . . . 3

1.3 Cách làm việc . . . 3

2 Kiến trúc hệ thống **4**

3 Kiến thức nền tảng **4**

3.1 Dữ liệu mất cân bằng và cách đo mô hình . . . 4

3.2 Chia dữ liệu theo thời gian và rò rỉ dữ liệu . . . 5

3.3 Logistic Regression . . . 6

3.4 Gradient Boosting . . . 6

3.5 Mã hoá theo nhãn (target encoding) . . . 7

3.6 Tái lập kết quả . . . 7
3.7 Mô phỏng dữ liệu thanh toán . . . 7
3.8 Phát hiện bất thường trên chuỗi KPI . . . 8
3.9 Kỹ thuật dữ liệu . . . 9

4 Product 02 — Rủi ro gian lận trên IEEE-CIS 9
4.1 Dữ liệu và phân tích khám phá . . . 9
4.2 Hai baseline (Stage B) . . . 10
4.3 Thí nghiệm rò rỉ dữ liệu (Stage E) . . . 11
4.4 Gradient Boosting (Stage E) . . . 12

5 Product 01 — Độ ổn định thanh toán 12
5.1 Dữ liệu giả lập (Stage C) . . . 13
5.2 Kết quả detector (Stage D) . . . 13

6 Dashboard 14

7 Đánh giá trung thực và hạn chế 15

8 Hướng phát triển 16

9 Kết luận 16

A Lệnh tái lập 17

B Thuật ngữ 17

1. Giới thiệu

1.1 Bối cảnh và câu hỏi

Vận hành thanh toán có hai câu hỏi thường trực: *hệ thống có đang chạy ổn không* (tỷ lệ thành công, độ trễ, lỗi theo đối tác) và *giao dịch này có đáng nghi không* (gian lận). Hai câu hỏi cần hai kiểu bài toán khác nhau: phát hiện bất thường trên chuỗi thời gian không có nhãn, và phân loại có nhãn trên dữ liệu rất mất cân bằng.

1.2 Mục tiêu và ràng buộc

- Làm trong 6 tuần, chạy cục bộ bằng Python và Docker (PostgreSQL 16).
- **Không dùng dữ liệu thật của công ty**, không lưu dữ liệu thẻ. Product 01 dùng dữ liệu tự sinh; Product 02 dùng dataset công khai IEEE-CIS theo điều khoản cuộc thi Kaggle.
- Người làm phải giải thích được mọi bước, nên ưu tiên phương pháp rõ ràng hơn phương pháp phức tạp.
- Chất lượng *đánh giá* (không rò rỉ, tái lập, báo cáo trung thực) quan trọng hơn điểm số.

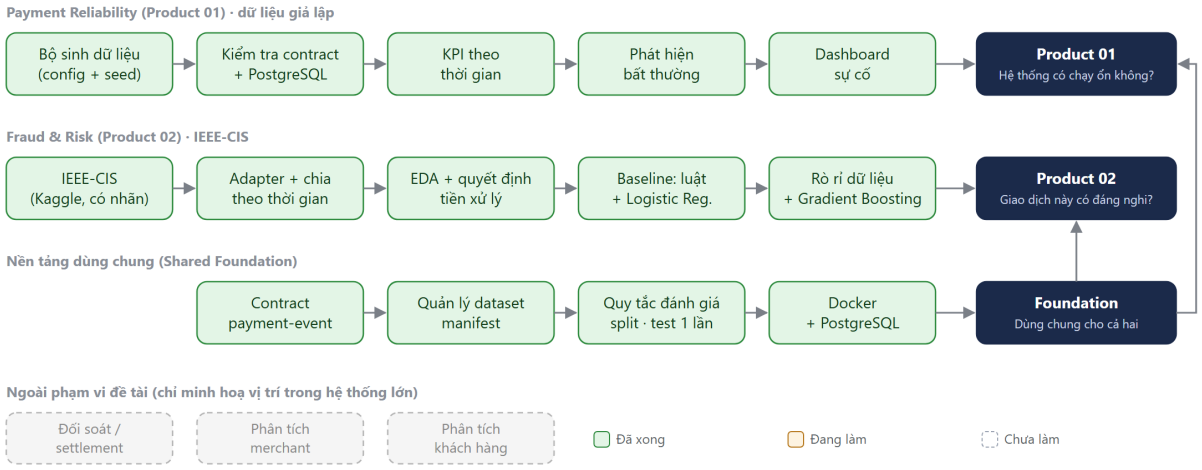
1.3 Cách làm việc

Mỗi giai đoạn bắt đầu bằng một bản đặc tả để chủ dự án duyệt, rồi mới viết code kèm kiểm thử. Quyết định kiến trúc được ghi thành ADR (Architecture Decision Record, [17]) trong `docs/decisions/` (ADR 0001–0007). Mỗi thay đổi đi qua một nhánh và một pull request; CI tự chạy `make lint` và `make test` (372 kiểm thử) trên Python 3.11 và 3.13.



Hình 1: Lộ trình 6 tuần theo giai đoạn A–F.

2. Kiến trúc hệ thống



Hình 2: Nền tảng dùng chung và hai sản phẩm. Hai sản phẩm dùng chung công cụ nhưng không dùng chung dữ liệu (ADR 0002).

Thành phần	Vai trò
Contract `payment-event` 1.0.0	JSON Schema mô tả một sự kiện thanh toán; mọi dữ liệu Product 01 phải qua kiểm tra này
Quản lý dataset	Manifest YAML ghi nguồn, giấy phép và checksum SHA-256 của từng file dữ liệu
Quy tắc đánh giá	Chia theo thời gian, test một lần, mã lần chạy tất định (mục 3.6)
PostgreSQL 16	Lưu sự kiện Product 01; KPI tính bằng SQL views (ADR 0005)
Dashboard	Trang HTML tĩnh tổng hợp kết quả hai sản phẩm (ADR 0007)

3. Kiến thức nền tảng

3.1 Dữ liệu mất cân bằng và cách đo mô hình

Trong IEEE-CIS chỉ khoảng 3,5% giao dịch là gian lận. Một mô hình luôn đoán "không gian lận" đạt độ chính xác (accuracy) 96,5% nhưng vô dụng. Vì vậy đề tài **không** dùng accuracy làm chỉ số chính.

Với một ngưỡng quyết định, mỗi giao dịch rơi vào một trong bốn ô: TP (gian lận, báo đúng), FP (hợp lệ, báo nhầm), FN (gian lận, bỏ sót), TN (hợp lệ, không báo).

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad \text{FPR} = \frac{FP}{FP+TN}, \qquad F_1 = \frac{2\,PR}{P+R}.$$

- **Precision**: trong các cảnh báo, bao nhiêu phần trăm đúng. **Recall**: trong các giao dịch gian lận, bắt được bao nhiêu.

- **PR-AUC** (average precision): diện tích dưới đường precision–recall khi quét mọi ngưỡng, AP $= \sum_n (R_n - R_{n-1}) P_n$. Mô hình ngẫu nhiên có PR-AUC bằng *tỷ lệ gian lận*, nên PR-AUC 0,165 trên tập có 3,44% gian lận nghĩa là gấp khoảng 4,8 lần ngẫu nhiên. Trên dữ liệu mất cân bằng, đường PR cho nhiều thông tin hơn đường ROC [1].

Giải thích kỹ: Recall tại FPR cố định

Precision phụ thuộc vào tỷ lệ gian lận: cùng một mô hình, tập nào nhiều gian lận hơn thì precision cao hơn. Vì validation (3,90%) và test (3,44%) có tỷ lệ khác nhau, so precision giữa hai tập dễ hiểu nhầm.
FPR chỉ tính trên giao dịch hợp lệ nên *không* phụ thuộc tỷ lệ gian lận. Chỉ số "recall tại FPR 5%" trả lời câu hỏi vận hành: *nếu chỉ chấp nhận làm phiền 5% khách hàng hợp lệ, ta bắt được bao nhiêu phần trăm gian lận?* Cách tính: quét các ngưỡng từ cao xuống thấp, giữ các ngưỡng có FPR $\leq 5\%$, lấy recall lớn nhất. Chỉ số này được khuyến nghị cho dữ liệu gian lận có thay đổi theo thời gian [2]. Mô hình ngẫu nhiên đạt đúng 0,05.

Giải thích kỹ: Khoảng tin cậy bootstrap cho PR-AUC

Một con số PR-AUC tính trên khoảng 118 nghìn dòng vẫn có sai số do chọn mẫu. Bootstrap ước lượng sai số đó mà không cần công thức:

1. Lấy ngẫu nhiên *có hoàn lại* n dòng từ n dòng của tập đánh giá (có dòng lặp, có dòng vắng).
2. Tính PR-AUC trên mẫu đó. Lặp lại $B = 200$ lần, với seed cố định để tái lập.
3. Khoảng tin cậy 95% là phân vị 2,5% và 97,5% của 200 giá trị.

Ví dụ: Logistic Regression trên test có PR-AUC 0,165 [0,155; 0,176]. Hai mô hình có khoảng tin cậy không chồng nhau thì khác biệt khó là do may rủi.

3.2 Chia dữ liệu theo thời gian và rò rỉ dữ liệu

Chia theo thời gian. Mô hình dùng thật luôn chấm điểm giao dịch *trong tương lai*. Vì vậy dữ liệu được xếp theo thời gian rồi chia 60% đầu làm train, 20% tiếp theo làm validation, 20% cuối làm test. Chia ngẫu nhiên sẽ cho mô hình "nhìn thấy tương lai" và làm kết quả lạc quan giả tạo. Hành vi gian lận thay đổi theo thời gian (*concept drift* [5]), nên điểm trên test thường thấp hơn trên validation; đề tài quan sát đúng điều này (mục 4).

Vai trò ba tập. Train để học. Validation để chọn ngưỡng và siêu tham số. Test để đo một lần cuối. Nếu nhìn kết quả test rồi chỉnh mô hình, test trở thành một tập validation thứ hai và mất giá trị.

Rò rỉ dữ liệu (data leakage) là khi mô hình dùng thông tin không có tại thời điểm dự đoán thật [3]. Các kiểu thường gặp [4]:

- *Không tách train/test đúng cách*: chia ngẫu nhiên với dữ liệu có thời gian; fit tiền xử lý (chuẩn hoá, điền thiếu) trên toàn bộ dữ liệu.
- *Đặc trưng mang nhãn*: một cột được tính từ nhãn của chính tập đánh giá.
- *Thông tin tương lai*: đặc trưng chỉ có sau thời điểm giao dịch.

Giải thích kỹ: Quy tắc test một lần và sổ đăng ký

Mỗi cấu hình có trường `status: draft` hoặc `frozen`. Lệnh chạy test (`make baseline-final`, `make gbm-final`, `make detect-report`) từ chối cấu hình chưa `frozen`. Khi chạy, nó tính một mã băm (hash) từ cấu hình và ghi vào sổ đăng ký (`artifacts/test_runs.json`). Lần chạy thứ hai với cùng hash bị từ chối, trừ khi dùng `--force`; khi đó cảnh báo được ghi thẳng vào kết quả. Muốn thử mô hình khác sau khi xem test thì phải tạo cấu hình mới, và báo cáo phải nói rõ.
Danh sách cấu hình được phép chạy test được *đăng ký trước* trong đặc tả. Ở Stage E chỉ có một: Gradient Boosting không có khoảng trống thời gian.

Giải thích kỹ: Độ trễ nhãn và khoảng trống thời gian (`gap_days`)

Trong thực tế, nhãn gian lận đến muộn: chargeback có thể mất vài tuần. Trong IEEE-CIS nhãn đã có sẵn, và theo mô tả của ban tổ chức, nhãn còn lan sang các giao dịch sau của cùng tài khoản. Nếu train kết thúc sát ngày validation bắt đầu, mô hình biết nhãn của cả những giao dịch mà ngoài đời chưa có nhãn.
Đề tài mô phỏng điều này bằng cách bỏ N ngày cuối của train ($N = 7, 14$) và giữ nguyên validation, test. Vì tập đánh giá không đổi, mọi khác biệt chỉ do khoảng trống. Đây là *phân tích độ nhạy*, không phải "giá trị đúng" (ADR 0003).

3.3 Logistic Regression

Mô hình tính xác suất gian lận

$$p(x) = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

và học w, b bằng cách tối thiểu hoá log-loss có trọng số:

$$L = -\sum_i c_{y_i} \big[y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i)) \big].$$

Với `class_weight='balanced'`, trọng số mỗi lớp là $c_k = \dfrac{n}{2\, n_k}$. Trên train: $c_1 \approx 14{,}78$ cho gian lận và $c_0 \approx 0{,}52$ cho hợp lệ, nên mô hình không bỏ qua lớp hiếm. Trước khi vào mô hình, cột số được điền thiếu bằng trung vị và chuẩn hoá $(x - \mu)/\sigma$; cột phân loại được one-hot. Cả trung vị, μ, σ và danh mục đều chỉ học trên train, trong một `Pipeline` của scikit-learn.

3.4 Gradient Boosting

Gradient Boosting cộng dần nhiều cây quyết định nhỏ:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x),$$

trong đó mỗi cây h_m được học để sửa sai của các cây trước (khớp với gradient âm của log-loss), và ν là tốc độ học (0,05 hoặc 0,1). Mô hình cây thường mạnh trên dữ liệu bảng [6], không cần chuẩn hoá, và tự xử lý giá trị thiếu.

`HistGradientBoostingClassifier` của scikit-learn chia giá trị mỗi cột thành tối đa 255 "thùng" (histogram) trước khi tìm điểm chia, nên nhanh trên vài trăm nghìn dòng. Đề tài tắt `early_stopping` vì phần validation nội bộ của nó chia *ngẫu nhiên*, trái quy tắc chia theo thời gian. Số cây (`max_iter`) nằm trong lưới 8 tổ hợp, chọn theo PR-AUC trên validation (ADR 0006).

3.5 Mã hoá theo nhãn (target encoding)

Cột `card1` (định danh thẻ) có hơn 10 nghìn giá trị, không one-hot được. Mã hoá theo nhãn thay mỗi giá trị v bằng tỷ lệ gian lận đã làm mượt:

$$\text{enc}(v) = \frac{\sum_{i:\,x_i=v} y_i + m \cdot \bar{y}}{n_v + m},$$

với $\bar{y}$ là tỷ lệ gian lận chung và $m = 100$ kéo các giá trị hiếm về $\bar{y}$.

Giải thích kỹ: Mã hoá ngoài fold (out-of-fold) và vì sao cần

Nếu mã hoá một dòng train bằng tỷ lệ gian lận có tính cả nhãn của *chính dòng đó*, đặc trưng "biết trước" đáp án trên train. Mô hình học cách tin vào đặc trưng này quá mức, rồi thất vọng trên validation. Trên dữ liệu thử nghiệm, cách làm này làm PR-AUC *giảm* từ 0,168 xuống 0,078.

Cách sửa: chia train thành 5 khối liên tiếp theo thời gian; mỗi khối được mã hoá bằng nhãn của 4 khối còn lại. Validation và test được mã hoá bằng nhãn của toàn bộ train. Khi đó không dòng nào dùng nhãn của chính nó, và PR-AUC trên dữ liệu thử về lại 0,166. Thí nghiệm rò rỉ (mục 4.3) so sánh bản này với bản cố tình dùng cả nhãn validation.

3.6 Tái lập kết quả

Giải thích kỹ: Mã lần chạy tất định (run id)

Mỗi lần chạy có mã `run-` + 12 ký tự đầu của SHA-256 tính trên: cấu hình dạng chuẩn hoá (bỏ trường `status`), seed, checksum của dataset và mốc chia dữ liệu. Cùng điều kiện thì cùng mã và cùng kết quả (có kiểm thử so từng byte của `metrics.json`). Đổi bất kỳ yếu tố nào thì ra mã mới. Mã git commit *không* nằm trong hash, vì sửa tài liệu không làm thay đổi một lần chạy. Bộ sinh dữ liệu dùng cùng ý tưởng cho `simulation_id`.

3.7 Mô phỏng dữ liệu thanh toán

- **Số giao dịch mỗi giờ** theo phân phối Poisson với cường độ $\lambda_h = 150 \times f_{\text{giờ}} \times f_{\text{ngày}}$; hệ số theo giờ Việt Nam (thấp ban đêm, cao buổi trưa và tối), trung bình bằng 1.
- **Số tiền và độ trễ** theo phân phối log-normal (lệch phải, luôn dương), tham số theo phương thức thanh toán và theo provider.
- **Lỗi và retry**: mỗi provider có xác suất lỗi riêng (15%, 22%, 30%, cố ý để cao). Chỉ lỗi kỹ thuật (`TIMEOUT`, `PROVIDER_ERROR`, `NETWORK_ERROR`) được thử lại; lỗi nghiệp vụ (`DECLINED`, `INVALID_REQUEST`) thì không.

- **Sự cố cài sẵn**: provider lỗi, độ trễ tăng, bùng lỗi mạng, mất lưu lượng một phương thức; mỗi loại có độ lớn (ví dụ x3 nghĩa là xác suất lỗi nhân 3). Nhãn sự cố chỉ nằm trong `anomaly_labels.json`, không có trong sự kiện.

Giải thích kỹ: Luồng số ngẫu nhiên riêng cho từng payment

Mỗi payment dùng một bộ sinh số ngẫu nhiên riêng, khởi tạo từ (seed, số thứ tự payment). Nhờ vậy, khi cài thêm một sự cố trong một cửa sổ thời gian, chỉ các payment chạm vào cửa sổ đó thay đổi; mọi payment khác giống hệt lần chạy không có sự cố. Kiểm thử xác nhận hơn 90% payment không đổi. Điều này cho phép so sánh "có sự cố" và "không có sự cố" trên *cùng* một thế giới giả lập.

3.8 Phát hiện bất thường trên chuỗi KPI

KPI được tính cho từng khung 15 phút và từng provider: tỷ lệ lỗi kỹ thuật, trung vị độ trễ, và tỷ trọng mỗi phương thức trong các payment mới. Ba KPI này ứng với ba trong bốn "tín hiệu vàng" của giám sát hệ thống (errors, latency, traffic) [9].

Giải thích kỹ: z-score bền vững (detector D-A, đường cơ sở)

z-score thông thường $(x-\mu)/\sigma$ bị kéo lệch khi trong lịch sử có vài giá trị cực đoan. Bản bền vững thay trung bình bằng *trung vị* và độ lệch chuẩn bằng *MAD* (trung vị của độ lệch tuyệt đối):

$$z_t = \frac{x_t - \text{median}(x_{t-96..t-1})}{1{,}4826 \cdot \text{MAD}(x_{t-96..t-1})}.$$

Hệ số 1,4826 làm MAD bằng độ lệch chuẩn khi dữ liệu có phân phối chuẩn. Lịch sử là 96 khung trước đó (24 giờ).
Điểm yếu: D-A không biết một tỷ lệ được tính trên bao nhiêu attempt. Ban đêm một khung chỉ có vài attempt, một lỗi trên hai attempt thành 50% và cho điểm rất cao. Kết quả thực nghiệm đúng như vậy: 14 lần báo nhầm của D-A đều rơi vào 01:45–05:45 giờ Việt Nam.

Giải thích kỹ: Điểm nhị thức gộp cửa sổ (detector D-B, phương án chính)

Ý tưởng: thay vì so *tỷ lệ*, so *số lỗi quan sát được* với số lỗi kỳ vọng, có tính đến số attempt.

1. Gộp $k = 2$ khung gần nhất (30 phút): có N attempt và K lỗi.
2. Lấy tỷ lệ lỗi nền p_0 từ 24 giờ trước cửa sổ đó.
3. Nếu không có sự cố, K xấp xỉ phân phối nhị thức $\text{Bin}(N, p_0)$, có trung bình Np_0 và độ lệch chuẩn $\sqrt{Np_0(1-p_0)}$. Điểm là số độ lệch chuẩn vượt kỳ vọng:

$$z = \frac{K - Np_0}{\sqrt{Np_0(1-p_0)}}.$$

Với cùng một mức tăng tỷ lệ, N nhỏ (ban đêm) cho z nhỏ, N lớn cho z lớn: detector tự "bớt tin" các khung ít dữ liệu. Với tỷ trọng phương thức (sự cố mất lưu lượng), K là số payment mới của phương thức đó, N là tổng payment mới; điểm được đổi dấu vì bất thường là *giảm*. Với độ trễ, D-B lấy trung bình log-trung vị có trọng số theo số attempt và chia cho độ phân

tán đã chuẩn hoá theo $\sqrt{n}$.
Vì sao ngưỡng phải cao (8 thay vì 3). Có 12 chuỗi KPI, mỗi chuỗi 672 khung trong 7 ngày, tức khoảng 8 nghìn lần kiểm tra. Ngưỡng 3 độ lệch chuẩn, nếu xét riêng từng lần, chỉ sai khoảng 0,13%, nhưng nhân với 8 nghìn lần là khoảng 10 lần báo nhầm trong 7 ngày. Lưới chỉnh ngưỡng xác nhận đúng như vậy: ngưỡng 3 báo nhầm 1,4–1,9 lần mỗi ngày, ngưỡng 8 không báo nhầm lần nào. Đây là vấn đề *kiểm định nhiều lần* (multiple testing).

Đánh giá theo sự cố. Các khung cảnh báo liền nhau (cách nhau tối đa 1 khung) được gộp thành một *sự cố*. Một bất thường cài sẵn được tính là "bắt được" nếu có sự cố *đúng đối tượng* (đúng provider, hoặc đúng phương thức) giao với cửa sổ của nó (cộng 15 phút ân hạn). Chỉ số báo cáo: số bất thường bắt được, độ trễ phát hiện, số sự cố báo nhầm mỗi ngày. Cách chấm theo cửa sổ dựa trên [12]; rủi ro "bất thường quá dễ" theo [11].

Giải thích kỹ: Tách bộ dữ liệu chỉnh ngưỡng và bộ báo cáo

Nếu chỉnh ngưỡng và đo kết quả trên cùng một bộ dữ liệu giả lập, ta chỉ chứng minh được rằng detector khớp với bộ đó. Vì vậy có hai bộ: *tuning* (seed 42) để chỉnh ngưỡng, và *report* (seed 2026) để đo một lần, có một loại sự cố mà bộ tuning không có (mất lưu lượng). Đây là phiên bản "train/test" cho bài toán không có nhãn thật.

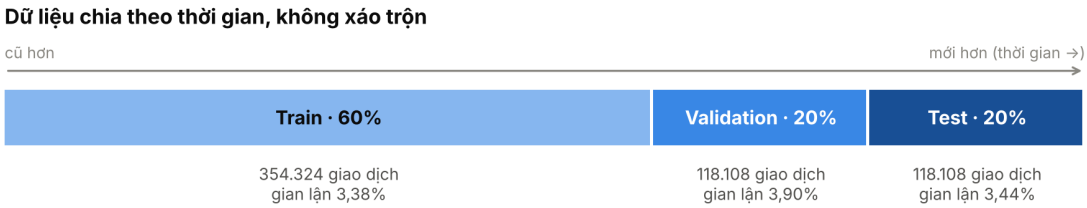
3.9 Kỹ thuật dữ liệu

- **Data contract** bằng JSON Schema [14]: quy định trường, kiểu, giá trị cho phép; cấm trường lạ (ví dụ `card_number`). Quy tắc mà schema không diễn đạt được (ví dụ `payment_id` khác `attempt_id`, các attempt của một payment phải cùng số tiền) được kiểm tra bằng code.
- **Ingestion idempotent**: nạp lại cùng file không tạo bản ghi trùng (khoá chính `event_id` và `INSERT … ON CONFLICT DO NOTHING`); file đã đổi nội dung thì bị từ chối. Thử nghiệm: nạp lại 30.050 sự kiện, thêm 0 dòng.
- **SQL views**: KPI là các câu SQL đọc được trực tiếp (`date_bin` để chia khung 15 phút, `percentile_cont` cho phân vị, `generate_series` để giữ cả khung trống). Kiểm thử so kết quả view với một cách tính độc lập bằng pandas.

4. Product 02 — Rủi ro gian lận trên IEEE-CIS

4.1 Dữ liệu và phân tích khám phá

590.540 giao dịch trong khoảng 6 tháng. Chia theo thời gian:



Hình 3: Chia dữ liệu theo thời gian, không xáo trộn.

Phân tích khám phá (EDA) chỉ chạy trên train và cho ra 9 quyết định tiền xử lý, mỗi quyết định có bằng chứng và được chủ dự án duyệt (`docs/data-analysis/eda-summary.md`). Ví dụ:

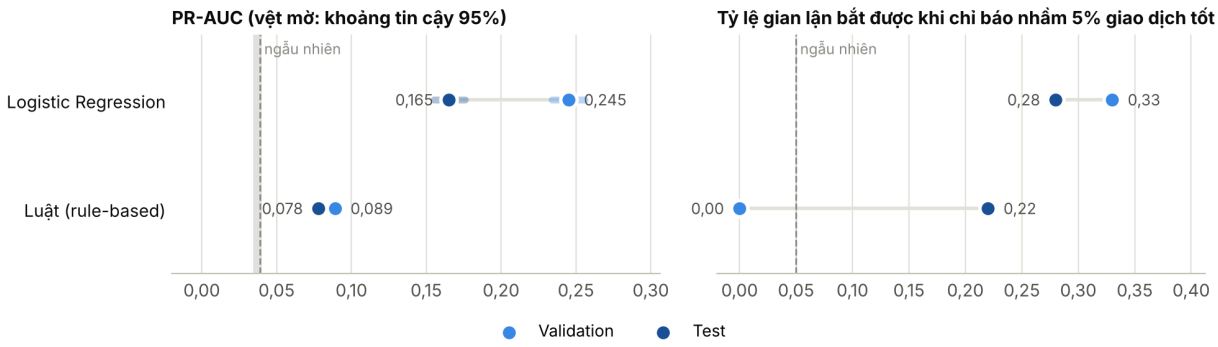
- `TransactionAmt` lệch phải rất mạnh (độ lệch 19,44; trung vị 70, lớn nhất 31.937), nên thêm $\log(1+x)$ (độ lệch còn 0,50).
- Fraud rate theo tuần trên train dao động 1,85%–4,55%, nên dùng `class_weight='balanced'` và chọn ngưỡng trên validation.
- Loại `TransactionID` và `TransactionDT` gốc vì chúng mã hoá thứ tự thời gian; chỉ dùng giờ trong ngày.

4.2 Hai baseline (Stage B)

Baseline luật: 3 luật minh bạch có ngưỡng học trên train (số tiền trên phân vị 99; `ProductCD` và loại thẻ có tỷ lệ gian lận cao); điểm là tỷ lệ luật bị kích hoạt. Logistic Regression: 20 đặc trưng (số tiền, log số tiền, giờ trong ngày, 14 cột đếm `C1`–`C14`, `ProductCD`, `card4`, `card6`).

Bảng 1: Kết quả lần chạy `run-d6faa4699008`, cấu hình đã `frozen`. Tập test mở một lần.

Mô hình	Tập	PR-AUC [95% CI]	Tỷ lệ gian lận	Recall @ FPR 5%
Logistic Regression	Validation	0,245 [0,234; 0,256]	3,90%	0,33
Logistic Regression	**Test**	**0,165** [0,155; 0,176]	3,44%	0,28
Luật	Validation	0,089 [0,084; 0,092]	3,90%	0,00
Luật	**Test**	**0,078** [0,074; 0,082]	3,44%	0,22



Hình 4: Validation (nhạt) và test (đậm) của hai mô hình. Vùng xám: mức của mô hình ngẫu nhiên.

Đọc kết quả. (1) Logistic Regression tốt hơn luật rõ rệt; khoảng tin cậy không chồng nhau. (2) Kết quả giảm từ validation sang test, kể cả chỉ số không phụ thuộc tỷ lệ gian lận (recall tại FPR 5%: 0,33 → 0,28), nên mức giảm không chỉ do tỷ lệ gian lận thấp hơn: dữ liệu thay đổi theo thời gian. (3) Số 0 của baseline luật trên validation là do điểm luật chỉ có vài mức rời rạc: mức chặt nhất báo nhầm 5,1% giao dịch tốt, vượt mục tiêu 5% một chút. (4) Có hai lần chạy test (`run-515a1190a0a4` trước khi thêm chỉ số FPR, và `run-d6faa4699008`); mọi chỉ số chung trùng khớp, nên lần thứ hai không dùng để chọn lại mô hình.

4.3 Thí nghiệm rò rỉ dữ liệu (Stage E)

Lấy đúng Logistic Regression của Stage B, mỗi biến thể phá *một* quy tắc, tất cả đo trên validation:

Biến thể	Thay đổi
Đúng quy tắc	Như Stage B (mốc so sánh)
Chia ngẫu nhiên	Xáo trộn trước khi chia 60/20/20
Tiền xử lý trên mọi dòng	Điền thiếu, chuẩn hoá, one-hot fit trên toàn bộ dữ liệu
Mã hoá `card1` hợp lệ	Thêm mã hoá theo nhãn, ngoài fold trên train
Mã hoá `card1` rò rỉ	Mã hoá bằng nhãn train *và validation*
Khoảng trống 7, 14 ngày	Bỏ cuối train (độ trễ nhãn)

Kèm bảng mô tả *thực thể lặp lại*: bao nhiêu phần trăm dòng validation và test có thẻ (`card1`, và tổ hợp `card1+addr1+P_emaildomain`) đã xuất hiện trong train. Tổ hợp ba cột là kinh nghiệm phổ biến trong các notebook công khai của cuộc thi; đề tài chỉ dùng ý tưởng, không sao chép code.

Bảng 2: Lần chạy `leak-827d929eaa73`, chỉ trên validation (118.108 dòng). Không đọc nhãn test.

Biến thể	PR-AUC [95% CI]	Chênh	Recall @ FPR 5%
Đúng quy tắc (Stage B)	0,245 [0,235; 0,257]	+0,000	0,33
Chia ngẫu nhiên	0,177 [0,168; 0,190]	−0,068	0,31
Tiền xử lý trên mọi dòng	0,244 [0,234; 0,256]	−0,001	0,33
Mã hoá `card1` hợp lệ	0,252 [0,242; 0,264]	+0,007	0,37
Mã hoá `card1` rò rỉ	0,308 [0,297; 0,321]	**+0,063**	0,43
Khoảng trống 7 ngày	0,244 [0,233; 0,256]	−0,002	0,32
Khoảng trống 14 ngày	0,241 [0,231; 0,253]	−0,004	0,32

Thực thể lặp lại: 98,7% dòng validation có `card1` đã gặp trong train; với `card1+addr1+P_emaildomain` là 85,9%.

Đọc kết quả. (1) Rò nhãn rõ nhất là mã hoá `card1` bằng nhãn validation: PR-AUC tăng 0,063, trong khi bản dùng nhãn train đúng cách chỉ tăng 0,007. (2) Tiền xử lý trên mọi dòng và

khoảng trống 7–14 ngày đổi PR-AUC không quá 0,004, nhỏ hơn độ rộng khoảng tin cậy (khoảng 0,02): với baseline này chúng không tạo khác biệt đo được. (3) **Ngược kỳ vọng**, chia ngẫu nhiên cho PR-AUC *thấp hơn* 0,068. Tỷ lệ gian lận của validation ngẫu nhiên thấp hơn (3,55% so với 3,90%) chỉ giải thích khoảng một phần ba mức giảm; khoảng 0,046 còn lại *chưa có lời giải thích*. Giả thuyết "validation rơi vào giai đoạn gian lận bùng phát" chỉ được hỗ trợ yếu, vì các tuần 4–10 của train cũng có tỷ lệ cao (3,8–4,6%).

4.4 Gradient Boosting (Stage E)

Cùng đặc trưng, cùng cách chia, cùng quy trình chọn ngưỡng với Logistic Regression, nên khác biệt chỉ đến từ mô hình. Lưới 8 tổ hợp (`learning_rate` 0,05/0,1; `max_iter` 200/400; `max_leaf_nodes` 15/31), chọn theo PR-AUC validation.

Tổ hợp được chọn: `learning_rate` 0,1, `max_iter` 400, `max_leaf_nodes` 15; ngưỡng 0,853 chọn trên validation (tối đa F1).

Bảng 3: Gradient Boosting `run-f8807cb6f780`, cấu hình `frozen` (commit `89a8ed5`), test mở một lần. Hai baseline lấy từ `run-d6faa4699008`.

Mô hình	PR-AUC validation	PR-AUC test	Recall @ FPR 5% (test)
Luật	0,089	0,078	0,22
Logistic Regression	0,245	0,165 [0,155; 0,176]	0,28
Gradient Boosting	0,497 [0,483; 0,514]	**0,408** [0,390; 0,424]	**0,51**

Đọc kết quả. (1) Trên test, Gradient Boosting cao gấp khoảng 2,5 lần Logistic Regression (0,408 so với 0,165); khoảng tin cậy không giao nhau. (2) Tại ngưỡng đã chọn: precision 0,516, recall 0,371, F1 0,432, FPR 1,24%. (3) PR-AUC giảm 0,089 từ validation sang test (Logistic Regression giảm 0,080): chọn tổ hợp và ngưỡng trên validation hơi lạc quan, và dữ liệu thay đổi theo thời gian. (4) Tổ hợp tốt nhất nằm ở cạnh lưới (`max_iter` = 400 là giá trị lớn nhất), nên chưa biết tăng thêm có tốt hơn không; mô hình chỉ dùng 20 đặc trưng, chưa dùng bảng identity.

5. Product 01 — Độ ổn định thanh toán

5.1 Dữ liệu giả lập (Stage C)

Bảng 4: Hai bộ mô phỏng 7 ngày, 3 provider, 20 merchant, 150 payment/giờ.

Chỉ số	Tuning (`sim-485cacc9c858`)	Report (`sim-f6e6749ff0bf`)
Payment / sự kiện	25.573 / 30.050	25.151 / 29.340
Payment có retry	15,0%	14,3%
Thành công ngay lần đầu	77,1%	77,6%
Thành công sau cùng	89,9%	90,1%
Độ trễ trung vị / p95 (ms)	579 / 1.509	582 / 1.522
Sự cố cài sẵn	4	4 (có loại mới)

Vì sao 150 payment/giờ thay vì 30 như kế hoạch ban đầu. Ở 30/giờ, mỗi provider chỉ có khoảng 2,5 attempt trong một khung 15 phút; tỷ lệ lỗi đo được dao động ngẫu nhiên khoảng ± 22 điểm %, lớn hơn chính tỷ lệ lỗi 15%. Ở 150/giờ, dao động còn khoảng ± 10 điểm %. (Độ lệch chuẩn của một tỷ lệ là $\sqrt{p(1-p)/n}$.)

Mỗi bộ có thêm một sự cố *yếu* (x1,5) để đo giới hạn phát hiện, vì sự cố mạnh (x3–x5) thì phương pháp nào cũng bắt được [11]. Cả hai bộ nạp vào PostgreSQL không có dòng bị từ chối.

5.2 Kết quả detector (Stage D)

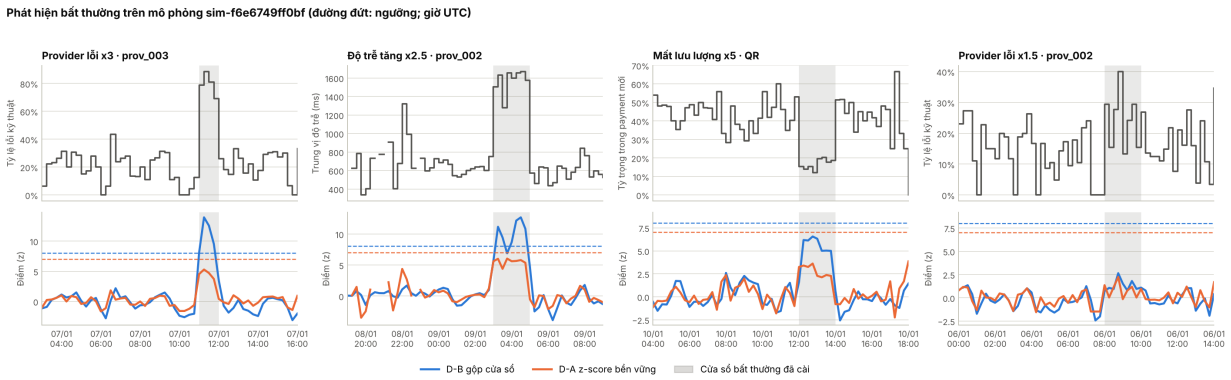
Ngưỡng được chọn bằng lưới trên bộ tuning theo quy tắc viết trước: bắt nhiều bất thường nhất với tối đa 1 báo nhầm mỗi ngày; hoà thì chọn độ trễ ngắn hơn rồi ngưỡng cao hơn. Kết quả: D-A ngưỡng 7; D-B ngưỡng 8, gộp 2 khung. Cấu hình được commit (`e331ec1`, 14:27:02 UTC) *trước* khi chạy bộ report (14:27:31 UTC).

Bảng 5: Tuning `det-3ddc26cff253`; report `det-08335c6af086` (một lần).

Detector	Tuning	Report
D-B gộp cửa sổ	3/4 sự cố, 0 báo nhầm/ngày	**2/4, 0 báo nhầm/ngày**
D-A z-score bền vững	2/4, 1,0 báo nhầm/ngày	0/4, 2,0 báo nhầm/ngày

Bảng 6: Từng sự cố trên bộ report. Điểm cao nhất D-A / D-B; ngưỡng 7 / 8.

Sự cố	Đối tượng	Điểm	D-A	D-B
Provider lỗi x3, 60 phút	`prov_003`	5,3 / 13,9	Trượt	Bắt, sau 15 phút
Độ trễ tăng x2,5, 2 giờ	`prov_002`	6,0 / 12,7	Trượt	Bắt, sau 30 phút
Mất lưu lượng QR x5, 2 giờ	QR	3,6 / 6,6	Trượt	**Trượt**
Provider lỗi x1,5, 2 giờ	`prov_002`	2,2 / 2,6	Trượt	Trượt



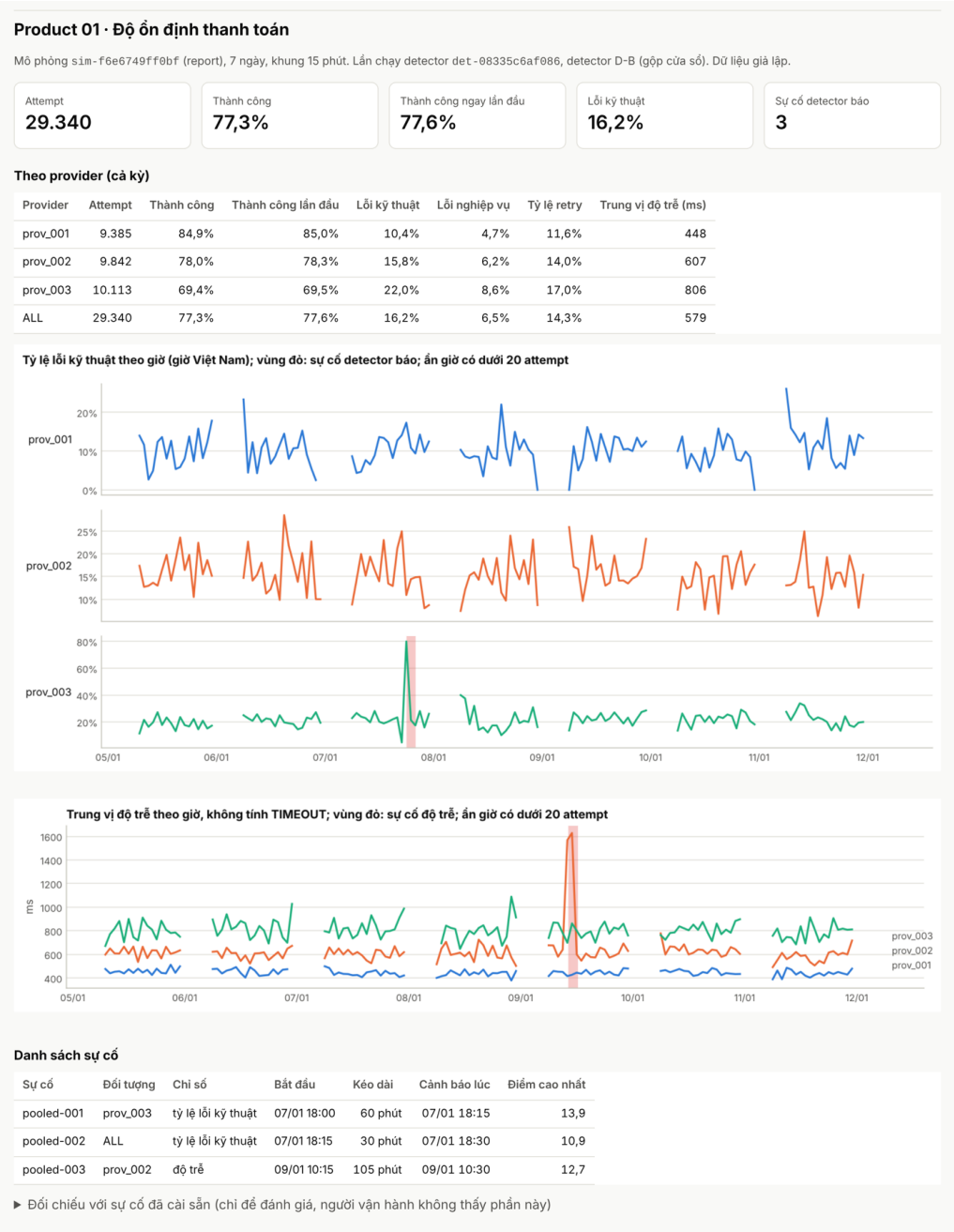
Hình 5: Bộ report: KPI của đúng đối tượng quanh mỗi sự cố (trên) và điểm của hai detector (dưới). Vùng xám: cửa sổ sự cố cài sẵn.

Đọc kết quả.

- **D-B tốt hơn D-A ở mọi mặt**, đúng lý do thiết kế: tính đến số attempt quan trọng hơn chọn ngưỡng khéo.
- **D-B trượt sự cố mất lưu lượng** dù nhìn bằng mắt rất rõ (tỷ trọng QR từ 44,0% xuống 16,6%): mỗi cửa sổ 30 phút chỉ có khoảng 110 payment mới, điểm đạt 6,6 dưới ngưỡng 8. Ngưỡng được chỉnh trên bộ tuning, nơi không có loại sự cố này. Đây chính là điều bộ report được thiết kế để lộ ra. Không chỉnh lại sau khi xem kết quả.
- **Sự cố x1,5 nằm dưới giới hạn phát hiện**: trên bộ report, tỷ lệ lỗi kỹ thuật của `prov_002` chỉ tăng từ 16,2% (24 giờ trước) lên 24,8% (35/141 attempt); gộp cả 2 giờ được khoảng 2,8 độ lệch chuẩn, thấp xa ngưỡng 8 cần để không báo nhầm. Trên bộ tuning cũng vậy (`prov_001`: 10,5% lên 15,0%).
- Mỗi bộ chỉ có 4 sự cố, nên đây là ví dụ minh hoạ, không phải tỷ lệ phát hiện.

6. Dashboard

`make dashboard` tạo một trang HTML tĩnh từ kết quả các lần chạy: không thêm thư viện, mở được không cần mạng, đính được vào báo cáo (ADR 0007). Phần Product 01 có ô số tổng, bảng theo provider, biểu đồ theo giờ Việt Nam có tô vùng sự cố, danh sách sự cố (cảnh báo `ALL` trùng thời gian với cảnh báo một provider được gộp vào sự cố của provider đó, vì đó chỉ là tiếng vọng), và một bảng đối chiếu với sự cố cài sẵn được tách riêng (người vận hành thật không có nhãn). Biểu đồ ẩn các giờ có dưới 20 attempt, vì tỷ lệ tính trên vài attempt chủ yếu là nhiễu, đúng cái bẫy làm D-A báo nhầm.



Hình 6: Phần Product 01 của dashboard trên bộ report.

7. Đánh giá trung thực và hạn chế

- **Test một lần**: mỗi cấu hình đã chốt chỉ mở tập test một lần; có sổ đăng ký chặn lần thứ hai. Danh sách cấu hình chạy test được đăng ký trước.
- **Dữ liệu Product 01 là giả lập**: mọi tham số do người làm đặt. Kết quả là "kiểm chứng trên bất thường đã biết", không phải hiệu năng thực tế.
- **IEEE-CIS**: nhãn lan theo tài khoản, cùng thẻ có thể xuất hiện ở cả train và test (ADR 0003). Chia theo thời gian không loại được hiện tượng này; thí nghiệm rò rỉ đo mức độ của nó.
- **Cỡ mẫu sự cố nhỏ**: 4 + 4 sự cố cài sẵn.
- **Giả định nhị thức của D-B** coi các attempt độc lập; retry của cùng payment không hoàn toàn

độc lập.

- **Tái lập dữ liệu giả lập phụ thuộc phiên bản numpy**: `uv.lock` chọn numpy khác nhau cho Python 3.11 và 3.13; nên dùng cùng phiên bản khi muốn khớp từng byte. Mã lần chạy không chứa phiên bản thư viện: chạy lại thí nghiệm rò rỉ trên máy khác (Python 3.12) ra cùng mã `leak-827d929eaa73` nhưng lệch $\pm$0,001 ở 3 dòng; Gradient Boosting và hai baseline khớp hoàn toàn.
- **Hạ tầng**: PostgreSQL mới được kiểm chứng trên một bản cài tạm; đường `make up` trên WSL chưa thử.
- **Stage E**: mức giảm PR-AUC khi chia ngẫu nhiên chỉ giải thích được một phần; Gradient Boosting chọn ở cạnh lưới và chỉ dùng 20 đặc trưng (mục 4.3, 4.4).

8. Hướng phát triển

1. **Giải thích kết quả chia ngẫu nhiên** của thí nghiệm rò rỉ, và mở rộng lưới Gradient Boosting (cần cấu hình mới và tập test mới).
2. **API chấm điểm** (đã cắt khỏi 6 tuần): nhận một giao dịch, trả điểm rủi ro và lý do; chỉ hỗ trợ quyết định, không tự động chặn.
3. **Ngưỡng riêng cho từng loại KPI**: chuỗi tỷ trọng ít nhiễu hơn chuỗi lỗi, nên một ngưỡng chung bỏ sót mất lưu lượng. Cần một bộ report mới để đánh giá trung thực.
4. **Detector có mùa vụ**: so với cùng giờ các ngày trước (phân rã chuỗi thời gian [10]), hoặc phương pháp dò thay đổi tích luỹ (CUSUM) để bắt sự cố yếu nhưng kéo dài.
5. **Mô hình và đặc trưng**: LightGBM; đặc trưng theo thẻ chỉ dùng thông tin quá khứ; hiệu chỉnh xác suất; giải thích mô hình.
6. **Nền tảng**: dbt thay SQL views khi số mô hình dữ liệu tăng; luồng sự kiện thời gian thực; theo dõi trôi dữ liệu.
7. **Dữ liệu thật** khi có phê duyệt, cùng quy trình đánh giá đã xây.

9. Kết luận

Trong 6 tuần, đề tài xây dựng nền tảng dùng chung (contract dữ liệu, quản lý dataset, quy tắc đánh giá, PostgreSQL) và hai sản phẩm có quy trình đánh giá giải thích được từng bước. Hai bài học rõ nhất: (1) với dữ liệu thay đổi theo thời gian, kết quả validation luôn lạc quan hơn test, nên phải giữ tập test cho một lần đo cuối; (2) với bài toán phát hiện bất thường, việc tính đến lượng dữ liệu phía sau mỗi con số quan trọng hơn việc chọn ngưỡng, và một bộ đánh giá độc lập có loại sự cố chưa gặp sẽ lộ ra những điểm mù mà bộ chỉnh ngưỡng không cho thấy.

A. Lệnh tái lập

Lệnh	Việc
`make setup`, `make lint test`	Cài môi trường, bật git hook; kiểm tra code
`make eda`, `make baseline`	EDA trên train; hai baseline trên validation
`make baseline-final`	Test một lần cho cấu hình đã chốt (chủ dự án)
`make simulate`, `make db-init`, `make ingest SIM=…`	Sinh và nạp dữ liệu giả lập
`make kpi SIM=…`, `make tune`, `make detect`	KPI, chỉnh ngưỡng, detector trên bộ tuning
`make detect-report`	Đánh giá một lần trên bộ report
`make leakage`, `make gbm`, `make gbm-final`	Stage E
`make dashboard`	Dashboard HTML

B. Thuật ngữ

Attempt	Một lần thử thanh toán; một payment có thể có nhiều attempt (retry)
Provider	Đối tác xử lý thanh toán
KPI	Chỉ số theo dõi, ở đây tính cho mỗi khung 15 phút
PR-AUC	Diện tích dưới đường precision–recall
FPR	Tỷ lệ giao dịch hợp lệ bị báo nhầm
Rò rỉ dữ liệu	Mô hình dùng thông tin không có lúc dự đoán thật
Concept drift	Quan hệ giữa dữ liệu và nhãn thay đổi theo thời gian
ADR	Tài liệu ghi một quyết định kiến trúc và lý do
`frozen`	Cấu hình đã chốt; chỉ chủ dự án đặt

Tài liệu

[1] T. Saito, M. Rehmsmeier. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10(3), 2015.

[2] S. Jesus et al. Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation. NeurIPS 2022 Datasets and Benchmarks, arXiv:2211.13358.

[3] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman. Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM TKDD* 6(4), 2012.

[4] S. Kapoor, A. Narayanan. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns* 4(9), 2023.

[5] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia. A Survey on Concept Drift Adaptation. *ACM Computing Surveys* 46(4), 2014.

[6] L. Grinsztajn, E. Oyallon, G. Varoquaux. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? NeurIPS 2022 Datasets and Benchmarks.

[7] Y.-A. Le Borgne, W. Siblini, B. Lebichot, G. Bontempi. *Reproducible Machine Learning for Credit Card Fraud Detection — Practical Handbook*. ULB, 2022.

[8] A. Dal Pozzolo et al. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE TNNLS* 29(8), 2018.

[9] B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.). *Site Reliability Engineering*. O'Reilly, 2016.

[10] R. J. Hyndman, G. Athanasopoulos. *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.

[11] R. Wu, E. Keogh. Current Time Series Anomaly Detection Benchmarks Are Flawed and Are Creating the Illusion of Progress. *IEEE TKDE*, 2021.

[12] A. Lavin, S. Ahmad. Evaluating Real-time Anomaly Detection Algorithms — the Numenta Anomaly Benchmark. ICMLA 2015.

[13] V. Chandola, A. Banerjee, V. Kumar. Anomaly Detection: A Survey. *ACM Computing Surveys* 41(3), 2009.

[14] JSON Schema Specification, bản 2020-12. `https://json-schema.org/specification`.

[15] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017.

[16] E. Lopez-Rojas, A. Elmir, S. Axelsson. PaySim: A Financial Mobile Money Simulator for Fraud Detection. EMSS 2016.

[17] M. Nygard. Documenting Architecture Decisions, 2011.

[18] IEEE-CIS Fraud Detection, Kaggle. `https://www.kaggle.com/c/ieee-fraud-detection`. Chỉ dùng cho nghiên cứu, theo điều khoản cuộc thi.